

TEST_STRATEGY_100_PERCENT

- [Helix Cluster OS — Test Strategy: 100% Coverage Per Test Type](#)
 - [Table of Contents](#)
- [Executive Summary](#)
 - [Current Test Inventory](#)
 - [Current Test Type Distribution](#)
 - [Missing Test Types \(11 of 20\)](#)
 - [Target](#)
- [Chapter 1: Test Type Definitions and Requirements](#)
 - [1.1 Unit Tests \(with Paired Mutation per Constitution §1.1\)](#)
 - [Definition](#)
 - [When to Use](#)
 - [Framework/Tool Requirements](#)
 - [Coverage Targets](#)
 - [Evidence Requirements \(per Constitution\)](#)
 - [Mutation Test Specification](#)
 - [1.2 Integration Tests \(Against REAL Services per CLAUDE-1\)](#)
 - [Definition](#)
 - [When to Use](#)
 - [Framework/Tool Requirements](#)
 - [Coverage Targets](#)
 - [Evidence Requirements](#)
 - [Integration Test Specification](#)
 - [1.3 E2E Tests \(End-User-Visible Behavior per CLAUDE-1\)](#)
 - [Definition](#)
 - [When to Use](#)
 - [Framework/Tool Requirements](#)
 - [Coverage Targets](#)
 - [Evidence Requirements](#)
 - [1.4 Chaos Tests \(25+ Fault Types per Phase 4\)](#)
 - [Definition](#)
 - [Fault Type Catalog](#)
 - [Coverage Targets](#)
 - [Evidence Requirements](#)
 - [1.5 Fuzz Tests \(Crypto, Parsing, Serialization\)](#)
 - [Definition](#)
 - [When to Use](#)
 - [Framework/Tool Requirements](#)
 - [Coverage Targets](#)
 - [Current Fuzz Test Inventory](#)
 - [Missing Fuzz Targets \(30+\)](#)
 - [1.6 Benchmark/Performance Tests](#)
 - [Definition](#)
 - [When to Use](#)
 - [Framework/Tool Requirements](#)
 - [Coverage Targets](#)
 - [Current Benchmarks](#)

- [1.7 Stress Tests \(Sustained Load, 48-Hour Soak\)](#)
 - [Definition](#)
 - [When to Use](#)
 - [Coverage Targets](#)
 - [Missing Implementation](#)
- [1.8 Mutation Tests \(Anti-Bluff per §1.1\)](#)
 - [Definition](#)
 - [Constitution Requirement](#)
 - [Coverage Targets](#)
 - [Current State](#)
 - [Required Mutation Tests by Package](#)
- [1.9 Hardening Tests \(Security, Boundary Conditions\)](#)
 - [Definition](#)
 - [When to Use](#)
 - [Coverage Targets](#)
 - [Current Hardening Tests](#)
- [1.10 Behavior Tests \(End-User Scenarios\)](#)
 - [Definition](#)
 - [When to Use](#)
 - [Coverage Targets](#)
- [1.11 DST \(Deterministic Simulation Testing\)](#)
 - [Definition](#)
 - [When to Use](#)
 - [Framework](#)
 - [Coverage Targets](#)
- [1.12 Compliance Tests \(Constitution Inheritance\)](#)
 - [Definition](#)
 - [When to Use](#)
 - [Coverage Targets](#)
- [1.13 Cross-Platform Tests \(Linux, macOS, Windows/WSL2\)](#)
 - [Definition](#)
 - [Constitution Requirement](#)
 - [Coverage Targets](#)
 - [Current Cross-Platform Tests](#)
- [1.14 GPU Backend Tests \(4 Vendor Backends per PCS-2\)](#)
 - [Definition](#)
 - [Coverage Targets](#)
 - [Current GPU Tests](#)
- [1.15 HelixQA Challenges \(per Constitution\)](#)
 - [Definition](#)
 - [Constitution Requirement](#)
 - [Coverage Targets](#)
- [1.16 Regression Tests \(Welch's t-test per Phase 4\)](#)
 - [Definition](#)
 - [Framework](#)
 - [Coverage Targets](#)
- [1.17 Load Tests \(50K Writes/sec per Phase 7 SLO\)](#)
 - [Definition](#)
 - [SLOs](#)
 - [Missing Implementation](#)
- [1.18 Soak Tests \(7-Day Sustained per Phase 8\)](#)
 - [Definition](#)

- [Coverage Targets](#)
 - [1.19 GameDay Tests \(Manual Fault Injection\)](#)
 - [Definition](#)
 - [Coverage Targets](#)
 - [1.20 Security Scanning \(govulncheck, gosec, trivy\)](#)
 - [Definition](#)
 - [Tools](#)
 - [Coverage Targets](#)
- [Chapter 2: Coverage Matrix](#)
 - [Package x Test Type Matrix](#)
- [Chapter 3: Missing Test Implementations](#)
 - [3.1 Missing Test Implementations by Type](#)
 - [Unit Tests \(Missing: ~50 packages\)](#)
 - [Mutation Tests \(Missing: ~200 packages\)](#)
 - [Integration Tests \(Missing: ~100 packages\)](#)
 - [E2E Tests \(Missing: ~30 features\)](#)
- [Chapter 4: Test Infrastructure Requirements](#)
 - [4.1 Test Runners and Frameworks](#)
 - [4.2 CI/CD Integration Plan](#)
 - [4.3 Test Environment Requirements](#)
 - [4.4 Test Data Management](#)
 - [4.5 Evidence Collection and Storage](#)
- [Chapter 5: HelixQA Integration](#)
 - [5.1 Challenge Runner Integration](#)
 - [5.2 Bank Creation for helix_cluster Features](#)
 - [Node Orchestration Challenges](#)
 - [Service Deployment Challenges](#)
 - [Health Monitoring Challenges](#)
 - [Security Challenges](#)
 - [5.3 Anti-Bluff Enforcement](#)
 - [5.4 Evidence Taxonomy](#)
- [Chapter 6: Implementation Plan](#)
 - [6.1 Prioritized Task List](#)
 - [Phase 0: Foundation \(This Wave\)](#)
 - [Phase 1: Dead-Code Resolution \(Owner-Gated\)](#)
 - [Phase 2: Test Coverage to Maximum](#)
 - [Phase 3: Challenges \(HelixQA\)](#)
 - [Phase 4: Responsiveness](#)
 - [Phase 5: Security Scanning](#)
 - [Phase 6: Autonomously-Actionable Backlog](#)
 - [Phase 7: Documentation](#)
 - [6.2 Per-Package Test Requirements](#)
 - [Template for New Tests](#)
 - [Priority Order for Test Implementation](#)
 - [6.3 Code-Level Specifications for New Tests](#)
 - [Specification: pkg/jwt Mutation Test](#)
 - [Specification: pkg/crypto Fuzz Test](#)
 - [Specification: pkg/swim Chaos Test](#)

Helix Cluster OS — Test Strategy: 100% Coverage Per Test Type

Field	Value
Document ID	TS-100PCT
Revision	1.0
Date	2026-03-04
Classification	INVESTIGATION — INTERNAL
Authors	Autonomous Test Strategy Agent
Status	FINAL
Constitution Reference	§1.1 (mutation-paired), §7.1 (quality), §11.4 (anti-bluff), CLAUDE-1 (end-user usability)

Table of Contents

- 1. [Executive Summary](#)
- 2. [Chapter 1: Test Type Definitions and Requirements](#)
- 3. [Chapter 2: Coverage Matrix](#)
- 4. [Chapter 3: Missing Test Implementations](#)
- 5. [Chapter 4: Test Infrastructure Requirements](#)
- 6. [Chapter 5: HelixQA Integration](#)
- 7. [Chapter 6: Implementation Plan](#)

Executive Summary

Current Test Inventory

Metric	Value
Total test files	~620
Total test functions	~4,907
Test types currently implemented	9 of 20
Packages with tests	~200 of 255

Metric	Value
Main module coverage	82.4%
Security module coverage	87.8%
Anti-bluff pass rate	33% (10/30 packages)
Mutation test coverage	~5 cases + 1 script

Current Test Type Distribution

Test Type	Count	Files	Constitution Requirement
Unit	~530	~400	§1.1 — paired with mutation
Integration	~64	~40	CLAUDE-1 — against REAL services
E2E	~7	~4	CLAUDE-1 — end-user-visible behavior
Benchmark	~10	~5	Phase 4 — hot paths
Chaos	~13	~4	Phase 4 — 25+ fault types
Fuzz	~8	~4	§11.4 — crypto + parsing
Hardening	~3	~2	§11.4 — security + boundary
Behavior	~3	~2	CLAUDE-1 — end-user scenarios
Mutation	1 script	1	§1.1 — anti-bluff

Missing Test Types (11 of 20)

Test Type	Current	Target	Gap
Stress	0	All hot paths	ALL missing
Soak	0	7-day sustained	ALL missing
Load	0	50K writes/sec	ALL missing
GameDay	0	Manual fault injection	ALL missing
DST	~2	All consensus subsystems	Partial
Compliance	~1	Constitution inheritance	Partial
Cross-Platform	~5	Linux + macOS + Win/WSL2	Partial

Test Type	Current	Target	Gap
GPU Backend	~3	4 vendor backends	Partial
HelixQA	0	Challenge execution	ALL missing
Regression	~1	Welch's t-test	Partial
Security Scan	1	govulncheck+gosec+trivy	Partial

Target

100% coverage per test type for ALL packages. This means every package must have at least one test of each applicable test type, with evidence collected per the Constitution's anti-bluff requirements.

Chapter 1: Test Type Definitions and Requirements

1.1 Unit Tests (with Paired Mutation per Constitution §1.1)

Definition

Unit tests verify individual functions and methods in isolation. Per Constitution §1.1, every unit test **MUST** have a paired mutation test that verifies the test would fail if the implementation were intentionally broken.

When to Use

- Every exported function and method
- Every critical internal function
- Every error path
- Every edge case and boundary condition

Framework/Tool Requirements

- Go standard `testing` package
- `testify/assert` and `testify/require` for assertions
- `go test -race` for race detection
- `go test -cover` for coverage measurement

Coverage Targets

- **Line coverage:** $\geq 80\%$ per package (enforced by `pkg/covgate`)

- **Function coverage:** 100% of exported functions
- **Mutation coverage:** Every unit test must have a paired `_Mutation` test

Evidence Requirements (per Constitution)

- Test output showing PASS
- Coverage report per package
- Mutation test showing the test catches the mutation
- `go test -race` showing no data races

Mutation Test Specification

Per §1.1, for each unit test `TestX`, there must be a corresponding `TestX_Mutation` that:

1. **Intentionally breaks the implementation** (e.g., removes a cap, changes a formula, inverts a condition)
2. **Runs the original test**
3. **Asserts the test FAILS** on the broken implementation
4. **Restores the original implementation**

Example:

```
func TestDuration(t *testing.T) {
    cfg := backoff.Default()
    got := cfg.Duration(5)
    assert.True(t, got <= cfg.MaxBackoff)
}

func TestDuration_Mutation(t *testing.T) {
    // Mutation: remove the cap
    original := backoff.Default().MaxBackoff
    backoff.Default().MaxBackoff = 0 // remove cap
    got := backoff.Default().Duration(5)
    // Original test would now fail – cap not enforced
    assert.True(t, got > original, "mutation: cap removed
        but test still passes")
    backoff.Default().MaxBackoff = original // restore
}
```

1.2 Integration Tests (Against REAL Services per CLAUDE-1)

Definition

Integration tests verify that components work correctly together against real services (not mocks). Per CLAUDE-1, mocks are permitted ONLY in unit tests per §11.4.27.

When to Use

- Service-to-service communication
- Database operations (PostgreSQL, etcd, Redis)
- External API calls (when possible)
- File system operations
- Network operations

Framework/Tool Requirements

- Build tag: `//go:build integration`
- Docker/Podman Compose for test dependencies
- `testcontainers-go` for ephemeral containers
- Real database connections (not mocks)

Coverage Targets

- Every `internal/` package must have ≥ 1 integration test
- Every database migration must have an integration test
- Every gRPC service must have an integration test against a real server
- Every external dependency must be tested with a real instance

Evidence Requirements

- Test output showing PASS against real service
- Service logs captured as evidence
- Database state verified before and after
- Network traces for communication verification

Integration Test Specification

```
//go:build integration

func TestSchedulerNodeIntegration(t *testing.T) {
    // Start real etcd
    ctx := context.Background()
    etcdClient := setupEtcd(t)
    defer etcdClient.Close()
```

```

// Start real scheduler
sched := scheduler.NewServer(etcdClient)
go sched.Start()
defer sched.Stop()

// Start real node
node := node.NewServer(etcdClient)
go node.Start()
defer node.Stop()

// Test real communication
conn, err := grpc.Dial("localhost:50052",
    grpc.WithInsecure())
require.NoError(t, err)
defer conn.Close()

client := pb.NewSchedulerServiceClient(conn)
resp, err := client.Schedule(ctx,
    &pb.ScheduleRequest{...})
require.NoError(t, err)
assert.NotEmpty(t, resp.JobId)
}

```

1.3 E2E Tests (End-User-Visible Behavior per CLAUDE-1)

Definition

End-to-end tests verify that complete user workflows work as an end user would experience them. Per CLAUDE-1, these tests **MUST** prove the feature works for end users, not just that code executes without panic.

When to Use

- Complete user workflows (submit job → get results)
- API endpoint chains (auth → submit → monitor → retrieve)
- Cross-service workflows (schedule → assign → execute → complete)
- Terminal session workflows (create → attach → resize → detach)

Framework/Tool Requirements

- Build tag: `//go:build e2e`
- Running cluster (Docker Compose)
- HTTP/gRPC client for API calls
- WebSocket client for streaming
- Terminal emulator for session tests

Coverage Targets

- Every REST endpoint must have an E2E test
- Every gRPC service must have an E2E test
- Every WebSocket stream must have an E2E test
- Every complete user workflow must have an E2E test

Evidence Requirements

- Full HTTP request/response captured
- gRPC request/response captured
- WebSocket message log
- Screenshot or terminal recording for UI
- Metrics showing operation succeeded

1.4 Chaos Tests (25+ Fault Types per Phase 4)

Definition

Chaos tests inject failures into running systems and verify correct behavior under fault conditions. Phase 4 requires 25+ fault types.

Fault Type Catalog

#	Fault Type	Target Package	Injection Method
1	Network partition	pkg/swim , pkg/leader	iptables / netem
2	Node crash	pkg/swim , internal/node	Process kill
3	Node slow	pkg/scheduler	CPU throttle
4	Disk full	internal/build	dd fill
5	Memory pressure	All services	cgroup limit
6	CPU starvation	All services	cgroup limit
7	Network latency	pkg/swim	tc netem delay
8	Network packet loss	pkg/swim	tc netem loss
9	Network reorder	pkg/swim	tc netem reorder
10	Network duplicate	pkg/swim	tc netem duplicate
11	etcd leader election	pkg/leader	Kill etcd leader

#	Fault Type	Target Package	Injection Method
12	etcd cluster shrink	pkg/leader	Remove etcd member
13	Redis connection loss	pkg/tieredcache	Kill Redis
14	PostgreSQL failover	internal/schema	Primary switch
15	gRPC connection drop	All services	Kill service mid-request
16	gRPC timeout	All services	Add latency
17	Certificate expiration	pkg/security	Expire cert
18	Clock skew	pkg/swim , pkg/crdt	Modify system clock
19	Process OOM	All services	cgroup oom
20	Disk I/O error	internal/build	Device error injection
21	DNS failure	Service discovery	DNS blackout
22	Concurrent leader election	pkg/leader	Multiple candidates
23	Split brain	pkg/splitbrain	Network partition
24	Hot loop / CPU spin	All services	Infinite loop injection
25	Memory leak simulation	All services	Gradual memory growth
26	Goroutine leak	All services	Blocked goroutine injection
27	Channel deadlock	All services	Channel fill
28	Unbounded queue	pkg/scheduler	Queue overflow

Coverage Targets

- At least 25 fault types tested
- Every consensus subsystem tested under partition
- Every stateful service tested under crash

- Every network operation tested under latency/loss

Evidence Requirements

- Fault injection log
- System behavior during fault
- Recovery verification after fault
- Metrics showing graceful degradation

1.5 Fuzz Tests (Crypto, Parsing, Serialization)

Definition

Fuzz tests provide random/mutated inputs to functions and verify they don't crash, hang, or produce incorrect results.

When to Use

- All crypto operations (hashing, signing, key exchange)
- All parsing operations (configuration, protocol, wire format)
- All serialization operations (JSON, protobuf, Avro, FlatBuffers)
- All network input handling

Framework/Tool Requirements

- Go native fuzzing (`go test -fuzz`)
- `go-fuzz` for more sophisticated fuzzing
- OSS-Fuzz integration for continuous fuzzing

Coverage Targets

- All crypto packages must have fuzz tests
- All parsing/serialization must have fuzz tests
- All protocol message handling must have fuzz tests

Current Fuzz Test Inventory

Package	Function	Status
<code>pkg/hybridkex</code>	<code>FuzzHybridKEX</code>	✓ Active
<code>pkg/x25519session</code>	<code>FuzzX25519Session</code>	✓ Active
<code>pkg/doublecrypt</code>	<code>FuzzDoubleCrypt</code>	✓ Active
<code>pkg/tracing</code>	<code>FuzzW3CTraceContext</code>	✓ Active
<code>pkg/covgate</code>	<code>FuzzParse</code>	✓ Active

Package	Function	Status
pkg/session	FuzzCRDTCheckpoint	✓ Active

Missing Fuzz Targets (30+)

1. pkg/crypto — Hash and key generation
2. pkg/jwt — Token parsing
3. pkg/security/tls.go — TLS configuration
4. pkg/security/vault.go — Vault secret handling
5. pkg/events/avro.go — Avro wire format
6. pkg/events/avro_wire.go — Avro wire deserialization
7. pkg/computeproto — FlatBuffers roundtrip
8. pkg/classads/parser.go — ClassAd expression parsing
9. pkg/config/config.go — Configuration parsing
10. pkg/netutil/cidr.go — CIDR parsing
11. pkg/openapivalidate — OpenAPI validation
12. internal/gateway/auth.go — Auth token parsing
13. internal/security/spiffe_ca.go — SPIFFE CA
14. pkg/wireguard/config.go — WireGuard config
15. pkg/swim/protocol.go — SWIM message parsing
16. pkg/resources/cgroup_v2.go — cgroup parsing
17. pkg/session/crdt.go — CRDT merge
18. pkg/etcd/keys.go — etcd key parsing
19. internal/gpu/nvidia_parser.go — NVIDIA SMI output parsing
20. pkg/storage/redis_client.go — Redis response parsing

1.6 Benchmark/Performance Tests

Definition

Benchmarks measure the performance of critical code paths and track performance over time.

When to Use

- All hot paths (scheduling, session management, event processing)
- All serialization paths
- All crypto operations
- All network operations

Framework/Tool Requirements

- Go standard `testing.Benchmark`
- `benchstat` for comparison
- `pprof` for profiling

Coverage Targets

- Every `internal/` package must have ≥ 1 benchmark
- Every hot-path function must have a benchmark
- Every serialization function must have a benchmark

Current Benchmarks

Package	Benchmark	Status
<code>pkg/scheduler</code>	Various	✓
<code>pkg/session</code>	Various	✓
<code>pkg/events</code>	Various	✓
<code>pkg/discovery</code>	Various	✓
<code>pkg/crdt</code>	Various	✓
<code>test/benchmark/gpu_bench_test.go</code>	GPU	✓
<code>test/benchmark/scheduler_bench_test.go</code>	Scheduler	✓
<code>test/benchmark/messaging_bench_test.go</code>	Messaging	✓
<code>test/benchmark/session_bench_test.go</code>	Session	✓
<code>test/benchmark/discovery_bench_test.go</code>	Discovery	✓

1.7 Stress Tests (Sustained Load, 48-Hour Soak)

Definition

Stress tests apply sustained load to the system and verify it remains stable and responsive.

When to Use

- All services under peak load
- All concurrent operations at maximum capacity
- All resource limits at boundary conditions

Coverage Targets

- 48-hour continuous operation without degradation
- Memory stability (no leaks)
- Goroutine stability (no leaks)

- Latency within SLO bounds

Missing Implementation

- No stress test framework exists
- No sustained load generator
- No memory/goroutine leak detection harness

1.8 Mutation Tests (Anti-Bluff per §1.1)

Definition

Mutation tests verify that unit tests actually catch bugs by intentionally breaking the implementation and checking that tests fail.

Constitution Requirement

Per §1.1, every unit test must have a paired mutation test. A test that passes on broken code is a PASS-bluff.

Coverage Targets

- 100% of unit tests must have paired mutation tests
- Every behavioral invariant must be verified by mutation

Current State

- Only ~5 explicit `Mutation` test cases exist
- `test/mutation/run_mutations.sh` exists but is limited
- Most packages have zero mutation coverage

Required Mutation Tests by Package

Package	Unit Tests	Mutation Tests	Gap
pkg/backoff	2	0	2
pkg/classads	3	0	3
pkg/config	2	0	2
pkg/context	2	0	2
pkg/crypto	2	0	2
pkg/events	1	0	1
pkg/grpcutil	2	0	2
pkg/health	1	0	1

Package	Unit Tests	Mutation Tests	Gap
pkg/infra	5	0	5
pkg/jwt	2	0	2
pkg/leader	1	0	1
pkg/log	21	8	13
pkg/lru	1	0	1
pkg/metrics	1	0	1
pkg/middleware	1	0	1
pkg/netutil	3	0	3
pkg/pubsub	1	0	1
pkg/ratelimit	1	0	1
pkg/retry	2	0	2
pkg/semaphore	1	0	1
pkg/serde	2	0	2
pkg/validator	2	0	2
pkg/websocket	1	0	1
pkg/workerpool	1	0	1
Total	~67	~8	~59

1.9 Hardening Tests (Security, Boundary Conditions)

Definition

Hardening tests verify that the system behaves correctly under adversarial or boundary conditions.

When to Use

- All security-sensitive operations
- All input validation
- All resource limits
- All error handling paths

Coverage Targets

- All authentication/authorization paths
- All input validation paths
- All resource exhaustion scenarios
- All error recovery paths

Current Hardening Tests

Package	Test	Status
internal/wireguard	hardening_test.go	✓
internal/wireguard	monitor_hardening_test.go	✓
internal/build	build_hardening_test.go	✓
internal/security	security_hardening_test.go	✓

1.10 Behavior Tests (End-User Scenarios)

Definition

Behavior tests verify that complete user-facing features work as expected from the end user's perspective.

When to Use

- Every feature claimed to be user-visible
- Every CLI command
- Every API workflow
- Every dashboard interaction

Coverage Targets

- Every feature in the MVP scope must have ≥ 1 behavior test
- Every CLI command must have a behavior test
- Every REST endpoint must have a behavior test

1.11 DST (Deterministic Simulation Testing)

Definition

DST uses deterministic simulation to test distributed system behavior under controlled fault injection. Inspired by FoundationDB's approach.

When to Use

- All consensus algorithms

- All distributed state management
- All leader election
- All session replication

Framework

- `pkg/testing/dst/engine.go` — DST engine
- `pkg/testing/dst/buggify.go` — Fault injection
- `pkg/testing/dstscale/scale.go` — Scale testing
- `pkg/testing/dstcompress/compress.go` — Timeline compression

Coverage Targets

- SWIM gossip convergence under churn
- Leader election under network partition
- CRDT convergence under concurrent updates
- Session state consistency under failure

1.12 Compliance Tests (Constitution Inheritance)

Definition

Compliance tests verify that the project adheres to all constitutional rules.

When to Use

- Every constitutional rule must be verified by a compliance test
- Every CLAUDE/AGENTS/QWEN rule must be verified
- Every anti-bluff rule must be verified

Coverage Targets

- All PCS rules (cross-platform parity)
- All §1.1 rules (mutation-paired tests)
- All §7.1 rules (end-user usability)
- All §11.4 rules (anti-bluff, no-hardcoding, no-sudo)

1.13 Cross-Platform Tests (Linux, macOS, Windows/WSL2)

Definition

Tests that verify features work on all supported platforms.

Constitution Requirement

Per PCS-1 (CLAUDE-2), every feature needs Linux, macOS, and Windows/WSL2 implementations.

Coverage Targets

- Every platform-specific file must be tested on its target platform
- Every build tag must have tests on the appropriate platform
- No blanket non-Linux skips

Current Cross-Platform Tests

Package	Linux	macOS	Windows
pkg/resources	✓ proc_linux.go	✓ proc_darwin.go	✗
pkg/wireguard	✓ kernel WG	✓ wireguard-go	✗
pkg/powergater	✓ reader_linux.go	✓ reader_darwin.go	✗
pkg/edgeheartbeat	✓ reader_linux.go	✓ reader_darwin.go	✗
internal/health	✓ syscall_unix.go	✓ syscall_unix.go	✓ syscall_windows.go
internal/gpu	✓ detect_linux.go	✓ detect_darwin.go	✗ detect_other.go

1.14 GPU Backend Tests (4 Vendor Backends per PCS-2)

Definition

Tests that verify GPU operations work across all 4 vendor backends: NVIDIA, AMD, Apple, Intel.

Coverage Targets

- NVIDIA: CUDA/PTX operations
- AMD: ROCm operations
- Apple: Metal operations
- Intel: oneAPI/SYCL operations

Current GPU Tests

Backend	Unit Tests	Integration Tests	Real Hardware
NVIDIA	✓	✓	Required
AMD	✗	✗	Required
Apple	✓	✓ (darwin)	Required
Intel	✗	✗	Required

1.15 HelixQA Challenges (per Constitution)

Definition

Challenges are executable verifications that prove a feature works for end users. They are the ultimate anti-bluff mechanism.

Constitution Requirement

Per CLAUDE-1, Challenges are bound equally to tests — a Challenge PASS on a broken feature is the same class of defect as a unit test PASS on broken code.

Coverage Targets

- Every user-facing feature must have ≥ 1 Challenge
- Every Challenge must capture sink-side evidence
- Every Challenge must prove end-user-visible operation

1.16 Regression Tests (Welch's t-test per Phase 4)

Definition

Regression tests detect performance degradation using statistical methods.

Framework

- `pkg/stats/welch.go` — Welch's t-test implementation
- `pkg/testing/regression/regression.go` — Regression framework

Coverage Targets

- All benchmarks must have regression tests
- All SLOs must have regression tests

- Performance changes must be detected with 95% confidence

1.17 Load Tests (50K Writes/sec per Phase 7 SLO)

Definition

Load tests verify that the system meets its performance SLOs under expected load.

SLOs

- **Write throughput:** $\geq 50,000$ writes/sec
- **Read latency:** $p99 \leq 10\text{ms}$
- **Schedule latency:** $p99 \leq 100\text{ms}$
- **Session attach:** $p99 \leq 50\text{ms}$

Missing Implementation

- No load testing framework
- No load generation tooling
- No SLO verification infrastructure

1.18 Soak Tests (7-Day Sustained per Phase 8)

Definition

Soak tests run the system under sustained load for extended periods (7 days) and verify stability.

Coverage Targets

- No memory leaks over 7 days
- No goroutine leaks over 7 days
- No performance degradation over 7 days
- No data corruption over 7 days

1.19 GameDay Tests (Manual Fault Injection)

Definition

GameDay tests are manual exercises where operators inject faults and verify the system's response.

Package	Unit	Mut	Int	E2E	Chaos	Fuzz	Bench	Stress	Soak	Hard
pkg/ crypto										
pkg/ discovery	✓	✓	✓	✗	✗	✗	✓	✗	✗	✗
pkg/ errors	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
pkg/ events	✓	✗	✓	✗	✗	✗	✓	✗	✗	✗
pkg/ grpcutil	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
pkg/ health	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
pkg/ hybridkex	✓	✗	✗	✗	✗	✓	✗	✗	✗	✗
pkg/infra	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
pkg/jwt	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
pkg/ leader	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
pkg/log	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗
pkg/lru	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
pkg/ metrics	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
pkg/ ratelimit	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
pkg/ resources	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
pkg/ scheduler	✓	✗	✓	✗	✓	✗	✓	✗	✗	✗
pkg/ security	✓	✗	✓	✗	✗	✗	✗	✗	✗	✓
pkg/ session	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗
pkg/swim	✓	✓	✓	✗	✓	✗	✗	✗	✗	✗

Package	Unit	Mut	Int	E2E	Chaos	Fuzz	Bench	Stress	Soak	Hard
pkg/tracing	✓	✗	✗	✗	✗	✓	✗	✗	✗	✗
pkg/wireguard	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
internal/gateway	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗
internal/session	✓	✗	✓	✓	✗	✗	✗	✗	✗	✗
internal/scheduler	✓	✗	✓	✗	✗	✗	✓	✗	✗	✗
internal/node	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
internal/security	✓	✗	✓	✗	✗	✗	✗	✗	✗	✓
internal/health	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
internal/build	✓	✗	✓	✗	✗	✗	✗	✗	✗	✓
internal/policy	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗
internal/gpu	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗

Legend: ✓ = Has tests, ✗ = Missing, ● = Partial

Chapter 3: Missing Test Implementations

3.1 Missing Test Implementations by Type

Unit Tests (Missing: ~50 packages)

1. pkg/anticheat — Token verification unit tests
2. pkg/attestadmit — Admission control unit tests
3. pkg/auditproof — Audit proof unit tests
4. pkg/billingfsm — FSM state transition tests

5. pkg/burstcapacity — Capacity calculation tests
6. pkg/bursthysteresis — Hysteresis logic tests
7. pkg/capability — Capability check tests
8. pkg/cellmesh — Mesh routing tests
9. pkg/chaosexp — Experiment definition tests
10. pkg/cloudspot — IMDS parsing tests
11. pkg/costrouter — Cost routing tests
12. pkg/costsched — Cost scheduling tests
13. pkg/costtracker — Cost tracking tests
14. pkg/databrowser — Data browsing tests
15. pkg/devicecatalog — Catalog management tests
16. pkg/deviceplugin — Plugin framework tests
17. pkg/deviceprofile — Profile management tests
18. pkg/edgefusion — Data fusion tests
19. pkg/edgeverify — Edge verification tests
20. pkg/epochresolve — Epoch resolution tests
21. pkg/ewmarank — EWM ranking tests
22. pkg/exportcontrol — Export control tests
23. pkg/failconfirm — Failure confirmation tests
24. pkg/fallbackchain — Fallback chain tests
25. pkg/fedtrust — Federation trust tests
26. pkg/fiber — Fiber admission tests
27. pkg/flowcontrol — Flow control tests
28. pkg/forecast — Resource forecasting tests
29. pkg/gpuattest — GPU attestation tests
30. pkg/gpucatalog — GPU catalog tests
31. pkg/gpupool — GPU pooling tests
32. pkg/gputopo — GPU topology tests
33. pkg/grafanadash — Dashboard generation tests
34. pkg/gravaladmit — GraVal admission tests
35. pkg/gravalverify — GraVal verification tests
36. pkg/helixnet — Network simulation tests
37. pkg/helixtask — Task management tests
38. pkg/hybridtco — TCO calculation tests
39. pkg/idempotent — Idempotency tests
40. pkg/imagepolicy — Image policy tests
41. pkg/inferenceproxy — Inference proxy tests
42. pkg/jobadmit — Job admission tests
43. pkg/kraft — Unikraft integration tests
44. pkg/latencysched — Latency scheduling tests
45. pkg/llmfailover — LLM failover tests
46. pkg/local — Local execution tests
47. pkg/marketplaceadapter — Marketplace adapter tests
48. pkg/metering — Metering tests
49. pkg/modelintegrity — Model integrity tests
50. pkg/modelretry — Model retry tests

Mutation Tests (Missing: ~200 packages)

Every package with unit tests needs paired mutation tests. The current deficit is approximately 200 packages.

Integration Tests (Missing: ~100 packages)

All `internal/` packages and critical `pkg/` packages need integration tests against real services.

E2E Tests (Missing: ~30 features)

Every user-facing feature needs an end-to-end test.

Chapter 4: Test Infrastructure Requirements

4.1 Test Runners and Frameworks

Framework	Purpose	Current Status	Required
<code>Go testing</code>	Unit/integration tests	✓ Active	—
<code>testify</code>	Assertions	✓ Active	—
<code>testcontainers-go</code>	Ephemeral containers	✗ Absent	Required
<code>go test -fuzz</code>	Fuzz testing	✓ Active	Expand
<code>go test -race</code>	Race detection	✓ Active	—
<code>go test -bench</code>	Benchmarking	✓ Active	Expand
<code>helixqa</code>	Challenge runner	⚠ Partial	Full integration
<code>pkg/testing/dst</code>	DST engine	⚠ Partial	Expand scenarios
<code>pkg/testing/chaos</code>	Chaos framework	⚠ Partial	Expand faults

4.2 CI/CD Integration Plan

Current: No CI (all workflows disabled).

Target: Full CI pipeline with all test types.

Phased approach: 1. Enable go-build.yml and go-test.yml first 2. Add integration test stage 3. Add chaos test stage 4. Add security scan stage 5. Add E2E test stage 6. Add benchmark comparison stage 7. Add coverage gate enforcement

4.3 Test Environment Requirements

Environment	Purpose	Resources
Local dev	Unit tests	Developer machine
Docker Compose	Integration tests	16GB RAM, 4 CPU
Kind/K3s	E2E tests	32GB RAM, 8 CPU
Linux CI runner	WireGuard + cgroup tests	Root access, 16GB RAM
macOS CI runner	Darwin-specific tests	Apple hardware
GPU CI runner	GPU backend tests	NVIDIA + AMD GPUs
Multi-host	Network chaos tests	3+ nodes

4.4 Test Data Management

Data Type	Source	Storage	Rotation
Fixtures	testdata/ directories	Git	Per release
Seed data	scripts/seed-data.sql	Git	Per release
Generated	Test factories	Ephemeral	Per test run
Captured	Evidence collection	qa-results/	Per wave

4.5 Evidence Collection and Storage

Per Constitution §11.4, all test evidence must be captured and stored:

```
qa-results/  
├ docs_chain/<run-id>/          # Documentation sync  
evidence  
├ security/<run-id>/            # Security scan results  
├ challenges/<run-id>/          # Challenge execution  
evidence  
├ coverage/<run-id>/            # Coverage reports  
├ benchmarks/<run-id>/          # Benchmark results  
├ chaos/<run-id>/                # Chaos test results
```

└─ soak/<run-id>/	# Soak test results
└─ regression/<run-id>/	# Regression test results
└─ compliance/<run-id>/	# Compliance test results

Chapter 5: HelixQA Integration

5.1 Challenge Runner Integration

The HelixQA framework (`cmd/helix-test`) provides challenge execution. Integration requires:

1. Challenge bank population for all `helix_cluster` features
2. Evidence collection pipeline
3. Anti-bluff enforcement per §11.4
4. Sink-side evidence capture

5.2 Bank Creation for `helix_cluster` Features

Node Orchestration Challenges

- `challenge-node-register` — Register a node and verify it appears in discovery
- `challenge-node-heartbeat` — Send heartbeats and verify health updates
- `challenge-node-gpu-report` — Report GPU devices and verify inventory

Service Deployment Challenges

- `challenge-build-submit` — Submit a build and verify completion
- `challenge-build-stream` — Stream build logs and verify output
- `challenge-build-cancel` — Cancel a build and verify cleanup

Health Monitoring Challenges

- `challenge-health-check` — Check health and verify response
- `challenge-health-watch` — Watch health stream and verify updates
- `challenge-health-aggregate` — Verify aggregate health rollout

Security Challenges

- `challenge-security-auth` — Authenticate and verify token
- `challenge-security-rbac` — Verify role-based access control
- `challenge-security-mtls` — Verify mutual TLS between services

5.3 Anti-Bluff Enforcement

Per CLAUDE-1, a Challenge PASS on a broken feature is the same class of defect as a unit test PASS on broken code. The HelixQA framework must:

1. Verify the feature actually works (not just that the API returns 200)
2. Capture sink-side evidence (actual data, not just status codes)
3. Run discrimination tests (slightly wrong input should fail)
4. Require mutation-paired tests for the feature

5.4 Evidence Taxonomy

Evidence Type	Format	Storage	Retention
Test output	Text	qa-results/	90 days
Coverage report	HTML + JSON	qa-results/ coverage/	90 days
Benchmark results	JSON	qa-results/ benchmarks/	Indefinite
Security scan	JSON + SARIF	qa-results/ security/	365 days
Challenge evidence	Markdown + screenshots	qa-results/ challenges/	Indefinite
Chaos results	JSON + logs	qa-results/ chaos/	90 days
Soak metrics	Prometheus format	qa-results/ soak/	365 days

Chapter 6: Implementation Plan

6.1 Prioritized Task List

Phase 0: Foundation (This Wave)

1.  Fix Makefile build target (HXC-1637)
2.  Implement real `helixctl` CLI (HXC-1638)
3.  Fix SQL schema drift (HXC-1639)
4.  Fix concurrency hazards F1-F7

Phase 1: Dead-Code Resolution (Owner-Gated)

1. Triage 178 orphaned packages into Wire-in / Prune / Document

2. Wire the control-plane spine (leader → etcd → CRDT → STONITH)
3. Wire scheduler helpers (backfill → priorityqueue → nodeselector)
4. Wire security extensions (attestadmit → gravaladmit → gpuattest)

Phase 2: Test Coverage to Maximum

1. Add mutation tests for all packages with unit tests (~200 packages)
2. Add integration tests for all `internal/` packages (~14 packages)
3. Add fuzz tests for crypto and parsing (~30 targets)
4. Add stress/chaos tests for consensus subsystems
5. Wire coverage gate (`pkg/covgate`)

Phase 3: Challenges (HelixQA)

1. Author Challenges for each wired feature
2. Execute Challenges with sink-side evidence
3. Verify anti-bluff enforcement
4. Store evidence in `qa-results/`

Phase 4: Responsiveness

1. Fix remaining concurrency hazards (F8-F10)
2. Add bounded worker pools
3. Add lazy initialization
4. Add monitoring/metrics-driven optimization tests

Phase 5: Security Scanning

1. Promote gosec to build gate
2. Add trivy container scanning
3. Add Snyc/SonarQube when tokens available
4. Wire SBOM into release artifacts

Phase 6: Autonomously-Actionable Backlog

1. Work ~78 Bucket-A items in dependency order
2. Each closes with real tests + Challenge

Phase 7: Documentation

1. Update all docs per CLAUDE-3
2. Run `docs_chain sync` and verify
3. Generate all exports (md → html/pdf/docx)

6.2 Per-Package Test Requirements

Template for New Tests

For each package `pkg/X` :

```

pkg/X/
├── X.go                # Implementation
├── X test.go           # Unit tests (with Mutation variants)
├── X integration_test.go # Integration tests (build tag:
integration)
├── X fuzz test.go      # Fuzz tests (if parsing/crypto)
├── X hardening test.go # Hardening tests (if security)
├── X behavior test.go  # Behavior tests (if user-facing)
├── X chaos test.go     # Chaos tests (if distributed)
└── testdata/          # Test fixtures

```

Priority Order for Test Implementation

1. **P0 — Security packages:** pkg/jwt, pkg/crypto, pkg/security, pkg/hybridkex
2. **P0 — Concurrency packages:** pkg/swim, pkg/session, pkg/scheduler, EventBus
3. **P1 — Core packages:** pkg/discovery, pkg/leader, pkg/etcd, pkg/events
4. **P1 — Scheduling packages:** pkg/scheduler, pkg/backfill, pkg/priorityqueue
5. **P2 — Infrastructure packages:** pkg/resources, pkg/wireguard, pkg/infra
6. **P2 — Utility packages:** pkg/config, pkg/log, pkg/errors, pkg/middleware
7. **P3 — All remaining packages**

6.3 Code-Level Specifications for New Tests

Specification: pkg/jwt Mutation Test

```

// pkg/jwt/package_test.go
func TestParse_Mutation(t *testing.T) {
    // Create a real signed token
    token := createSignedToken(t, "user123", "admin")

    // Verify Parse succeeds on valid token
    parsed, err := jwt.Parse(token)
    require.NoError(t, err)
    assert.Equal(t, "user123", parsed.Claims.Subject)

    // Mutation: tamper with signature
    parts := strings.Split(token, ".")
    tampered := parts[0] + "." + parts[1] + "." +
        base64.RawURLEncoding.EncodeToString([]byte("tampered"))
    _, err = jwt.Parse(tampered)
    assert.Error(t, err, "mutation: tampered token should
    fail verification")
}

```

```

// Mutation: expired token
expired := createExpiredToken(t, "user123")
_, err = jwt.Parse(expired)
assert.Error(t, err, "mutation: expired token should
    fail verification")
}

```

Specification: pkg/crypto Fuzz Test

```

// pkg/crypto/package_test.go
func FuzzHash(f *testing.F) {
    f.Add([]byte(""))
    f.Add([]byte("hello"))
    f.Add([]byte{0x00})

    f.Fuzz(func(t *testing.T, data []byte) {
        hash := crypto.Hash(data)
        if len(hash) != 32 {
            t.Fatalf("hash length = %d, want 32", len(hash))
        }
        // Verify determinism
        hash2 := crypto.Hash(data)
        if !bytes.Equal(hash, hash2) {
            t.Fatal("hash is not deterministic")
        }
        // Verify different inputs produce different hashes
        if len(data) > 0 {
            modified := make([]byte, len(data))
            copy(modified, data)
            modified[0] ^= 0x01
            hash3 := crypto.Hash(modified)
            if bytes.Equal(hash, hash3) {
                t.Fatal("different inputs produced same
                    hash")
            }
        }
    })
}

```

Specification: pkg/swim Chaos Test

```

//go:build chaos

func TestSWIMPartitionRecovery(t *testing.T) {
    // Create 5-node cluster
}

```

```

nodes := makeCluster(t, 5)
defer nodes.Stop()

// Verify all nodes see each other
require.Equal(t, 5, len(nodes[0].HealthyMembers()))

// Partition: isolate nodes 3 and 4
nodes.Partition([]int{0, 1, 2}, []int{3, 4})

// Wait for suspicion timeout
time.Sleep(5 * time.Second)

// Verify majority partition continues
assert.Equal(t, 3, len(nodes[0].HealthyMembers()))

// Heal partition
nodes.Heal()

// Wait for convergence
time.Sleep(10 * time.Second)

// Verify full membership restored
assert.Equal(t, 5, len(nodes[0].HealthyMembers()))
}

```

End of Test Strategy Document

Document Statistics: - Total test types defined: 20 - Total test type requirements: 20 × 4 attributes each = 80 requirements - Total missing test implementations enumerated: 300+ - Total packages in coverage matrix: 35 - Total prioritized tasks: 7 phases - Total code-level specifications: 3